



American Spend
Elastic Markets Audit Report

Contents

[Findings Summary](#)

[High](#)

[Medium](#)

[Low](#)

[Informational](#)

Introduction

A security review of the AmericanSpend protocol was done, focusing on the security aspects of the smart contracts. This audit was performed by [33Labs](#).

Audit Team

Auditor	Role
Radoslav Radev ↗	Security Researcher
Samuel Troy Dominguez ↗	Security Researcher

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

About 33Labs

33Labs is an independent smart contract security research and development group with six years of experience and over 40 completed audits. We specialize in building and auditing Solidity, Rust, Move, and DeFi systems. Learn more at 33labs.ai or follow us on X [@33labs](#).

About AmericanSpend

AmericanSpend is a decentralized protocol featuring pool-based architecture, fee management, vault system, liquidity management.

Repository: [americanspend/markets](https://github.com/americanspend/markets)

Commit: 49ee06e4

Scope: americanspend/markets/*

Severity Definitions

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - The technical, economic, and reputation damage from a successful attack

Likelihood - The chance that a particular vulnerability gets discovered and exploited

Severity - The overall criticality of the risk

Informational - Findings in this category are recommended changes for improving the structure, usability, and overall effectiveness of the system.

Findings Summary

ID	Title	Severity	Status
[H-01]	USDC-blacklisted trader with active buy order permanently bricks graduated market finalization, locking all funds	High	Fixed
[H-02]	Push USDC Refunds During Finalization Can Permanently Brick Resolution	High	Fixed
[H-03]	<code>claimYield</code> and resolution payouts both pull from the same collateral balance	High	Fixed
[M-01]	Seed fund collateral locked permanently after graduation - Phase 1 users lose seed fee contributions on cancel/resolve	Medium	Fixed
[M-02]	Resolution delay can be reset by anyone trading on the CLOB	Medium	Fixed
[M-03]	Vault Yield Permanently Locked on Market Cancellation	Medium	Fixed
[M-04]	Graduation blocked when tapered ladders fall below <code>minOrderSize</code> (runtime + factory gap)	Medium	Fixed
[M-05]	Zero vault redemption snapshot breaks idempotency on the second <code>_finalizeVaultAccounting</code> call	Medium	Fixed
[M-06]	Cancel without refund sends only <code>totalPool + seedFund</code> , not full redeemed vault balance	Medium	Fixed
[M-07]	Vault redeem revert can permanently block market lifecycle finalization	Medium	Fixed
[M-08]	<code>buyOnPhase1</code> slippage check protects the wrong direction for buyers	Medium	Fixed
[M-09]	<code>marketBuy</code> slippage guard excludes taker fee, allowing total spend to exceed <code>maxCollateralIn</code>	Medium	Fixed
[M-10]	Seed ask-token flow increases winning-token denominator without adding to <code>payoutPool</code>	Medium	Fixed
[L-02]	Complete-set mint when <code>amount</code> is smaller than outcome count	Low	Fixed
[L-03]	<code>cancelAllOrders</code> resets the resolution-delay clock	Low	Fixed
[L-05]	<code>levelsPerSide * levelsPerSide</code> can overflow before ladder logic runs	Low	Fixed
[L-06]	Empty bid ladder leaves that outcome's <code>bidAlloc</code> collateral in the Market after <code>seedFund</code> is cleared	Low	Fixed
[L-07]	<code>graduate()</code> can execute after market close while CLOB placement is already closed	Low	Fixed
[L-08]	<code>calculatePotentialPayout</code> uses raw token supply instead of excluding market-held tokens	Low	Fixed
[L-09]	Stale <code>pools[]</code> after cancel-without-refund causes misleading view returns	Low	Fixed

ID	Title	Severity	Status
[L-10]	<code>closeEarly()</code> leaves <code>timing.closeTime</code> stale, breaking <code>getMarketData()</code> consistency	Low	Fixed
[L-11]	<code>MarketOrderExecuted</code> event reports pre-fee notional for buys but post-fee proceeds for sells	Low	Fixed
[L-12]	Cancel-no-refund zeroes <code>totalPool</code> but keeps <code>pools[]</code> , causing misleading <code>calculatePayout()</code>	Low	Fixed
[L-13]	Serial <code>cancelOrder</code> calls can delay resolution past intended window	Low	Fixed
[L-14]	Serial <code>cancelOrder</code> calls can delay resolution past intended window	Low	Fixed
[I-01]	Append-Only FIFO Queues Let Attackers Inflate Taker Matching Gas	Informational	Fixed
[I-02]	Transfer lock can be bypassed by burn-style transfer to <code>address(0)</code>	Informational	Fixed
[I-03]	<code>redeem()</code> makes redundant cross-contract call for winner balance	Informational	Fixed
[I-04]	<code>positionCount</code> increments on <code>burnCompleteSet</code> , making it an operation counter rather than a position counter	Informational	Fixed
[I-05]	Extremely low tick/size combinations can produce zero-notional matches (EDGE CASE)	Informational	Fixed

High

[H-01] USDC-blacklisted trader with active buy order permanently bricks graduated market finalization, locking all funds

Description

When a graduated market is resolved or cancelled, the `processFinalizationBatch` function iterates through every CLOB order sequentially via `cancelOrdersRange`, cancelling each and refunding reserves to the order's `trader`. For buy orders, the refund is USDC sent via `collateralToken.safeTransfer(recipient, refundCollateral)`.

USDC has a blacklist. If the `trader` address of any active buy order gets blacklisted by Circle, that `safeTransfer` reverts. There's no try-catch, no skip mechanism and no fallback recipient. The batch reverts entirely and the cursor never advances past the stuck order.

The market then enters a permanent deadlock. Every possible admin action is blocked:

- `processFinalizationBatch()` - reverts on the blacklisted transfer, every time
- `resolve()` / `cancel()` - revert with `InvalidFinalizationState` (finalization already pending)
- `scheduleEmergencyWithdraw(collateralToken)` - reverts with `MarketStillActive` (market is neither resolved nor cancelled)
- `closeEarly()`, `pause()`, `cancelAllOrders()`, all setters - revert with `FinalizationInProgress`
- User calling `cancelOrder()` directly - same `safeTransfer` to blacklisted address, same revert

Market contracts are minimal clones. Not upgradeable. No escape hatch exists.

The vulnerable code in both CLOB implementations (`CLOBRBTTree.sol`, `CLOBLinkedList.sol`):

```
if (order.isBuy) {
  _decrementActiveBuyLevel(order.priceBps);
  refundCollateral = reservedCollateralByOrderId[orderId];
  reservedCollateralByOrderId[orderId] = 0;
  if (refundCollateral > 0) {
    collateralToken.safeTransfer(recipient, refundCollateral); // @audit reverts if blacklisted
  }
}
```

Impact

- Total permanent loss of ALL collateral in the market - `totalPool` plus any untracked seed fund collateral. No user can ever redeem. Affects every participant in the market, not just the blacklisted user.

Recommendations

Wrap the refund transfer in a try-catch. On failure, escrow the funds to a mapping that the user (or an admin) can pull later and continue processing the next order. Something like:

```
if (refundCollateral > 0) {
  try collateralToken.transfer(recipient, refundCollateral) returns (bool success) {
    if (!success) {
```

```

        failedRefunds[recipient] += refundCollateral;
    }
} catch {
    failedRefunds[recipient] += refundCollateral;
}
}

```

Status

Acknowledged / Accepted Risk (Not Fixed)

GitHub issue #5 is closed, but the comments and later commit history do **not** support treating this as fixed. The team acknowledged it as a real failure mode and discussed whether to prioritize a non-reverting refund path or document it as an explicit USDC blacklist dependency / accepted deployment risk. A later referencing commit explicitly says the failed-refund escrow approach was reverted and documented as [wontfix](#) / [KNOWN_ISSUES](#).

[H-02] Push USDC Refunds During Finalization Can Permanently Brick Resolution

Description

During graduated-market finalization, `Market.processFinalizationBatch` asks each CLOB to cancel resting orders in batches via `cancelOrdersRange`. For every buy order, `CLOB._cancelActiveOrder` refunds escrowed collateral with `collateralToken.safeTransfer(maker, amount)` — a push to the maker’s address.

USDC (and similar collateral) implements address-level blocklists: `transfer(to)` reverts when `to` is blacklisted. If any active buy order’s `trader` is blacklisted, that refund reverts, the whole `cancelOrdersRange` call reverts, and the batch never completes. The market stays in `pendingFinalizationType != NONE`, while `resolve` / `cancel` / `pause` / `cancelAllOrders` and other admin paths that use `onlyNotFinalizing` are blocked. There is no on-chain “skip this order” or “abort finalization” escape hatch.

Important User collateral escrowed inside the CLOB is not on the Market balance; `Market.scheduleEmergencyWithdraw` for the collateral token still requires `resolved || cancelled` (`_validateCollateralEmergencyWithdraw`), so it does not recover CLOB escrows while stuck. Non-collateral emergency paths do not move CLOB-held USDC either.

1. Buy-order cancellation — push USDC to maker

```

function _cancelActiveOrder(uint256 orderId, address recipient)
    internal
    returns (uint256 refundCollateral, uint256 refundTokens)
{
    IOrderBook.Order storage order = orders[orderId];
    order.active = false;
    if (order.isBuy) {
        _decrementActiveBuyLevel(order.priceBps);
        refundCollateral = reservedCollateralByOrderId[orderId];
        reservedCollateralByOrderId[orderId] = 0;
        if (refundCollateral > 0) {
            collateralToken.safeTransfer(recipient, refundCollateral);
        }
    }
}

```

If `recipient` (the resting buy's `trader`) cannot receive USDC, `safeTransfer` reverts and the entire transaction reverts. Solidity rolls back `order.active = false`, so the order stays active for the next attempt — same failure repeats.

Sell cancellations refund outcome tokens (ERC-6909) to the maker, not USDC, so Circle's USDC blacklist does not apply to that leg.

2. Batch cancellation — no try/catch, cursor only advances on success

```
while (orderId < finalOrderId && processed < maxOrders) {
    IOrderBook.Order storage order = orders[orderId];
    if (order.active) {
        address trader = order.trader;
        (uint256 refundCollateral, uint256 refundTokens) =
            _cancelActiveOrder(orderId, trader);
        emit OrderCancelled(
            orderId, trader, refundCollateral, refundTokens
        );
    }
    unchecked {
        ++orderId;
        ++processed;
    }
}
```

When `_cancelActiveOrder` reverts, execution never reaches `++orderId`. The factory's `processFinalizationBatch` transaction fails; `pendingFinalizationBookIndex` / per-book cursors are not updated on success because the whole outer call reverted.

3. Admin / lifecycle lock while finalization is pending

`Market.cancelAllOrders`, `pause`, `unpause`, `closeEarly`, and several config setters use `onlyNotFinalizing`, which reverts when `pendingFinalizationType != NONE`:

```
function _onlyNotFinalizing() internal view {
    if (pendingFinalizationType != IMarket.PendingFinalizationType.NONE) {
        revert IMarket.FinalizationInProgress();
    }
}
```

So the intended recovery tool (`cancelAllOrders`) is unavailable exactly when you are stuck in the batch finalization state. `resolve` / `cancel` also reject a second entry while pending is set.

`scheduleEmergencyWithdraw` is not gated by `onlyNotFinalizing`, but for `token == collateralToken` the schedule/execute path still requires the market to be finalized and past the claim window — useless for escaping this stuck state:

```
function _validateCollateralEmergencyWithdraw(address token) internal view {
    if (token != address(collateralToken)) return;
    if (!resolved && !cancelled) revert IMarket.MarketStillActive();
    if (actualResolutionTime == 0) revert IMarket.MarketNotResolved();
}
```

```

    if (block.timestamp <= actualResolutionTime + COLLATERAL_CLAIM_PERIOD) {
        revert IMarket.ClaimPeriodNotEnded();
    }
}

```

Impact

- For users and liquidity providers
- Funds tied to Market accounting (`totalPool` , etc., after vault finalization semantics) and all CLOB escrows (buy collateral + sell token locks) remain unusable until finalization completes. With a reverting refund, completion may never occur.
- For operators
- Resolution and cancellation workflows depend on `processFinalizationBatch` succeeding. One blacklisted buy maker (sanctions, Circle policy, compromised address flagged by issuer) can halt the entire pipeline across all books processed in order, not only the affected book's later batches.

Recommendations

1. Move finalization and cancellation refunds to a pull model: credit `pendingCollateralRefunds[trader]` in storage, mark the order inactive without transferring, and let makers call `claimCollateralRefund()` (with clear events). Batch finalization must never depend on a third-party USDC transfer succeeding.
2. `try / catch` around `safeTransfer` and on failure route the amount into the same pending mapping (same outcome as pull).

Status

Duplicate / Covered by H-01 (Not Separately Fixed)

GitHub issue #8 was closed after the team agreed it was a duplicate of issue #5 / H-01. It should not be presented as independently fixed. Treat it as duplicate coverage of the same USDC push-refund / blacklist finalization risk described in H-01.

[H-03] `claimYield` and resolution payouts both pull from the same collateral balance

Description

At resolution, `_finalizeVaultAccounting` redeems all tracked vault shares into `collateralToken` on the Market. `payoutPool` is then set from `totalPool` (after any loss waterfall) minus a resolution rake. Separately, `claimYield` lets vault participants withdraw `(resolvedVaultAssetsSnapshot * userShares / totalVaultSharesTracked) - principal` (plus rake on that gross yield), but only for users whose share value exceeds principal . users underwater get `NothingToClaim` and never “return” value to the pool.

Those yield transfers and `redeem()` payouts both use `collateralToken.safeTransfer` from the same contract balance. There is no invariant tying `payoutPool + Σ yield claims` to that balance. When vault price per share moved between deposits, early depositors can have positive “yield” that is economically redistribution from later depositors, not net vault profit. If positive gross yields sum to more than net vault surplus over pooled principal, whoever calls `claimYield` first can leave the contract unable to honor full `redeem()` amounts computed from `payoutPool` .

1. Yield claims transfer from the shared ERC20 balance

```
function claimYield() external nonReentrant {
    if (!resolved) revert IMarket.MarketNotResolved();
    if (yieldClaimed[msg.sender]) revert IMarket.YieldAlreadyClaimed();
    if (
        address(vault) == address(0) || totalVaultSharesTracked == 0
        || resolvedVaultAssetsSnapshot == 0
    ) revert IMarket.NothingToClaim();

    uint256 userShares = userVaultShares[msg.sender];
    if (userShares == 0) revert IMarket.NothingToClaim();

    uint256 userAssetsAtResolution = Math.mulDiv(
        resolvedVaultAssetsSnapshot,
        userShares,
        totalVaultSharesTracked,
        Math.Rounding.Floor
    );
    uint256 userPrincipal = userPrincipalDeposited[msg.sender];
    if (userAssetsAtResolution <= userPrincipal) {
        revert IMarket.NothingToClaim();
    }

    uint256 grossYield = userAssetsAtResolution - userPrincipal;
    (uint256 rake, uint256 netYield) =
        _computeRakeAndNet(grossYield, orderBookSeedConfig.rakePercentage);
    if (netYield == 0) revert IMarket.NothingToClaim();

    yieldClaimed[msg.sender] = true;

    if (rake > 0) {
        collateralToken.safeTransfer(treasury, rake);
    }
    collateralToken.safeTransfer(msg.sender, netYield);

    emit YieldClaimed(msg.sender, grossYield, rake, netYield);
}
```

2. Resolution fixes `payoutPool` from pool accounting, not from “remaining after yield”

```
function _finalizeResolvedMarket(uint8 correctOutcome_) internal {
    correctOutcome_ = correctOutcome_;
    uint256 internalWinningBalance = OutcomeToken(outcomeToken)
        .balanceOf(address(this), correctOutcome_);
    winningTokenSupplyAtResolution = OutcomeToken(outcomeToken)
        .totalSupply(correctOutcome_) - internalWinningBalance;
    resolved = true;
    actualResolutionTime = block.timestamp;
    _finalizeVaultAccounting();

    uint256 totalPoolValue = totalPool;
    uint256 correctPool = pools[correctOutcome_];
    uint256 rake = (totalPoolValue * orderBookSeedConfig.rakePercentage)
        / BPS_DENOMINATOR;

    payoutPool = correctPool == 0 ? 0 : totalPoolValue - rake;

    if (!graduated && seedFund > 0) {
```

```

        collateralToken.safeTransfer(treasury, seedFund);
        seedFund = 0;
    }

    if (correctPool == 0) {
        if (totalPoolValue > 0) {
            collateralToken.safeTransfer(treasury, totalPoolValue);
        }
    } else if (rake > 0) {
        collateralToken.safeTransfer(treasury, rake);
    }

    emit MarketResolved(correctOutcome_, actualResolutionTime);
}

```

3. Winners redeem against `payoutPool` from the same balance

```

function redeem() public nonReentrant {
    _validateRedeemPreconditions();
    uint256 payout = calculatePayout(msg.sender);
    if (payout == 0) revert IMarket.NothingToClaim();

    claimed[msg.sender] = true;
    _burnOutcomeTokensOnRedeem();
    collateralToken.safeTransfer(msg.sender, payout);

    emit Redeemed(msg.sender, payout);
}

```

```

if (!resolved) return 0;
if (pools[correctOutcome] == 0) return 0;

uint256 userWinningTokens =
    OutcomeToken(outcomeToken).balanceOf(user_, correctOutcome);
if (userWinningTokens == 0) return 0;
uint256 totalWinningTokens = winningTokenSupplyAtResolution;
if (totalWinningTokens == 0) return 0;

return (userWinningTokens * payoutPool) / totalWinningTokens;
}

```

4. Concrete shortfall sketch (two depositors, flat vault NAV)

Assume two outcomes, `rakePercentage = 500` (5%), no seed fund, no loss waterfall: `redeemedAssets == totalPool == 2000`, `resolvedVaultAssetsSnapshot = 2000`, `payoutPool = 1900` after resolution rake. Alice deposited at a lower share price (many shares); Bob at a higher price (fewer shares); both principals 1000. Alice's share of snapshot is 1333 → gross yield 333; Bob's is 667 → no yield. If Alice claims first (~333 out), the contract holds ~1667 while a sole winner's `redeem` can still require up to 1900 from `payoutPool` math—insufficient unless net vault profit (or different ordering) fills the gap.

Impact

User-facing: First-come-first-served: early `claimYield` can cause later `redeem` (or the reverse if `redeem` runs first and consumes balance) to revert for lack of ERC20 balance. **Economic:** Misalignment between share-based “yield” and **pooled** resolution accounting.

Recommendations

Enforce a global cap so aggregate yield claims cannot exceed actual vault profit over tracked principal (e.g. `max(0, resolvedVaultAssetsSnapshot - totalPrincipalDeposited)` or similar baseline), and distribute only that surplus pro-rata—or move yield to a separate vault / escrow funded only from realized surplus. Alternatively, settle yield only after payout obligations are reserved (e.g. hold `payoutPool` in a separate accounting transfer or compute yield on residual explicitly).

Status

Fixed

GitHub issue #17 is closed and is supported by explicit fixing commits referencing the issue, including commits titled `Cap aggregate yield claims at realized vault surplus. Fix #17` and `Cap aggregate vault yield at realized surplus`.

Medium

[M-01] Seed fund collateral locked permanently after graduation - Phase 1 users lose seed fee contributions on cancel/resolve

Description

During `Market.sol#graduate()` function, the seed fund is consumed: half goes to CLOBs as bid collateral, the other half backs minted ask-side tokens. `seedFund` is set to 0.

When the market later finalizes (resolve or cancel), `processFinalizationBatch` cancels all CLOB orders. Seed bid orders return their collateral to Market (the seed order `trader`). Seed ask orders return tokens to Market. The bid collateral plus the ask-fund portion (which never left Market) reconstitute roughly the full original seed fund value in Market's balance.

But nothing tracks this returned collateral. `totalPool` was never updated. `seedFund` is 0. The collateral sits in the contract invisible to all payout logic.

For `cancel(refund=true)`, `calculatePayout` reads `seedFund` (which is 0) and computes `outcomeBacking = pools[i]` - no seed fund distribution. Users get back only their pool contributions.

For non-graduated cancellations, `seedFund > 0` and users get `pools[i] + (seedFund * pools[i]) / totalPool` - the full amount including seed fees.

This is an asymmetry. Phase 1 users paid seed fee surcharges (up to 10% per the tier config). After graduation, those fees are non-recoverable on cancellation.

Concrete example:

- Phase 1 deposits: \$110,000 total (at 10% seed fee tier)
- `totalPool = $100,000`, `seedFund = $10,000`
- Market graduates, then gets cancelled with refund
- After finalization: Market balance = \$110,000 (seed collateral fully returned from CLOBs)
- `calculatePayout` distributes only \$100,000 (`seedFund = 0`)
- \$10,000 permanently locked

Impact

- Phase 1 users lose their seed fee contributions whenever a graduated market is cancelled with refund or resolved. The locked amount equals the full seed fund value. For markets with high seed fee tiers and large pools, this can be significant (up to 10% of total Phase 1 deposits based on `MAX_RAKE_BPS` validation on seed fee tiers).
- The same issue exists at resolution - `payoutPool = totalPool - rake` doesn't include returned seed collateral. The collateral from filled seed orders (CLOB users buying seed ask tokens) also goes to Market untracked, compounding the locked amount.

Recommendations

Track the returned seed fund value after finalization and include it in the payout/refund calculations. The simplest fix: after all finalization batches complete but before calling `_finalizeResolvedMarket` or `_finalizeCancelledMarket`, compute the excess collateral in Market beyond `totalPool` and either add it back to `seedFund` or send it to treasury:

```
// After all CLOB orders cancelled, before final resolution/cancellation:
uint256 excessCollateral = collateralToken.balanceOf(address(this)) - totalPool;
if (excessCollateral > 0) {
    collateralToken.safeTransfer(treasury, excessCollateral);
}
```

Another mitigation is for cancel-with-refund, restore `seedFund` so users get their contributions back.

Status

Fixed — **radeveth**: severity depends on the devs answer from the slack question about seed fund refund intent

[M-02] Resolution delay can be reset by anyone trading on the CLOB

Description

`resolve` won't run until `resolutionDelay` has passed since the latest “activity” timestamp the contract knows about. For graduated markets that includes every CLOB's `lastActivityTimestamp`, and any public `placeLimit*` / `market*` call bumps that to `block.timestamp`.

So while the book is still accepting orders, someone can keep placing small trades (or doing anything that touches `_recordActivityTimestamp` on the CLOB) and push the window forward every time you get close to resolving. That's annoying for winners and ops, and it's cheap on L2.

It's not a “stall forever with no countermove” button: `closeEarly()` and `pause()` stop new CLOB orders (`closed` / `paused` in `_enforceOrderPlacementStatus`) without moving the clock to “now” the way a new trade does. Practical playbook is freeze first (close or pause), wait `resolutionDelay` from the last trade before the freeze, then `resolve`. The grieving window is “how long the market stayed open after the last meaningful trade,” not unbounded if ops uses those levers.

1. Where `resolve` enforces the delay

```
uint256 latestActivityTimestamp = _getLatestActivityTimestamp();
if (latestActivityTimestamp > 0 && timing.resolutionDelay > 0) {
    if (
        block.timestamp
        < latestActivityTimestamp + timing.resolutionDelay
    ) revert IMarket.MustWaitAfterLastTrade();
}
```

2. How “latest activity” is defined

```
function _getLatestActivityTimestamp()
    internal
    view
    returns (uint256 latestTimestamp)
{
    latestTimestamp = phase1LastActivityTimestamp;
```

```

uint256 orderBooksLength = outcomeOrderBooks.length;
for (uint256 i = 0; i < orderBooksLength; i++) {
    uint256 orderBookActivityTimestamp =
        IOrderBook(outcomeOrderBooks[i]).lastActivityTimestamp();
    if (orderBookActivityTimestamp > latestTimestamp) {
        latestTimestamp = orderBookActivityTimestamp;
    }
}
}

```

So the clock is the max of Phase 1 activity and every outcome book.

3. CLOB mutating paths bump the book timestamp

```

function _recordActivityTimestamp() internal {
    lastActivityTimestamp = block.timestamp;
}

```

That gets hit from matching, placement, cancellation batches, etc. — anything that's supposed to count as “there was action on this book.”

4. Why `closeEarly` / `pause` matter

`closeEarly` only sets `closed` and emits — it does not call `_recordActivityTimestamp`:

```

function closeEarly() external onlyFactory onlyNotFinalizing {
    if (closed || resolved || cancelled) revert IMarket.AlreadyFinalized();
    closed = true;
    emit MarketClosed();
}

```

CLOB entry points check `marketState.closed()` (and `paused()`):

```

if (
    marketState.closed()
    || (closeTime != 0 && block.timestamp >= closeTime)
    || (closeTime == 0
        && resolutionTime != 0
        && block.timestamp >= resolutionTime)
) revert IOrderBook.MarketClosed();

```

Once the market is closed or paused, new orders shouldn't land, so the max CLOB timestamp stops moving. Your delay baseline is the last update before that freeze.

Impact

Winners / LPs

- Resolution and payout flow starts later than admins might expect if nobody freezes trading first. **Griefers**
- Cost is basically gas + minimal escrow per poke. on Base that's usually small. Motivation is political / spite / slowing a rival, not draining the contract through this path alone.
- I wouldn't describe this as “all TVL permanently frozen”, it's **resolution blocked** until delay logic is satisfied. users can still cancel resting orders, etc, while the market is open.

Recommendations

1. Document “before `resolve` on a graduated market: `closeEarly` or `pause`, then wait `resolutionDelay` from last on-chain activity, then `resolve` (or start batched finalization).”
2. Optionally snapshot activity at close time (single storage slot) so `resolve` keys off a frozen value instead of live CLOB clocks, removes the whole class of “one more trade resets the timer” after you’ve already decided to wind down.

Status

Fixed — melanke: Good point I don’t think this creates a major issue in practice, but the cumulative effect is real. The admin has a way to mitigate it: they can call `closeEarly` and then `cancelAllOrders` to clear the book in a single step before resolving, so there will be only one delay after `cancelAllOrders`, instead of pushing the delay over and over. That said, if this turns out to be undesirable in practice, we could consider adjusting the delay rules so that cancellations after `closeEarly` no longer affect the resolution window.

[M-03] Vault Yield Permanently Locked on Market Cancellation

Description

`claimYield()` in `Market.sol` gates on `if (!resolved) revert MarketNotResolved()`. Cancelled markets never set `resolved = true` - the two flags are mutually exclusive via the `AlreadyFinalized` guard. So users can never call `claimYield()` on a cancelled market.

The problem is that `_finalizeCancelledMarket()` still calls `_finalizeVaultAccounting()`, which redeems all ERC-4626 vault shares and deposits the full principal + accrued yield back into the market’s collateral balance. When there’s yield (redeemed assets exceed principal), `_applyVaultLossWaterfall()` returns immediately without adjusting `totalPool` or `pools[]`. The yield just sits in the contract untracked.

`calculatePayout()` for cancelled markets computes refunds purely from `pools[i]` and `seedFund` - neither includes vault yield. So users collectively receive back their principal, while the yield above principal has no distribution path. It’s only recoverable by the factory owner via `executeEmergencyWithdraw` 90 days after cancellation.

Impact

- Users lose all vault yield earned during the market’s lifetime whenever a market with a configured vault is cancelled. This applies to both graduated and non-graduated markets. A 1M USDC market running 6 months at 4% APY silently forfeits ~20,000 USDC in yield that users rightfully earned.

Recommendations

Allow `claimYield()` when the market is cancelled:

```
function claimYield() external nonReentrant {
    if (!resolved && !cancelled) revert MarketNotResolved();
    ...
}
```

Status

Fixed — melanke: Confirmed. `_finalizeCancelledMarket` redeems all vault shares via `_finalizeVaultAccounting`, but `_applyVaultLossWaterfall` early-returns when there is positive yield, so `pools[]` and `seedFund` never absorb it. `calculatePayout` then refunds only `totalPool + seedFund`, and `claimYield` is gated on `!resolved`, so the yield

has no distribution path.

[M-04] Graduation blocked when tapered ladders fall below `minOrderSize` (runtime + factory gap)

Description

Runtime: Seed ladders weight levels with $L^2 - k^2$. The outermost level gets weight $2L - 1$ while `wSum` is $\Theta(L^3)$, so its share of `askFund / n` or bid collateral scales roughly as $\Theta(1/L^2)$. For moderate `seedFund`, large `orderAmount` (so `levelsPerSide = orderAmount / 2`), large `minOrderSize`, or more outcomes, the last level's floored token amount can fall below `minOrderSize` → `InsufficientSeedFund` → the whole `graduate()` reverts and the market stays in Phase 1. `orderBookSeedConfig` is fixed at deploy; there is no post-deploy retune.

Factory: `MarketFactory` validates seed-fee tiers and order-book fields separately. `minCoverage` only enforces price span $((orderAmount >> 1) * liquidityGap)$, not that accumulated `seedFund` at `graduationThreshold` can satisfy every ladder level at `minOrderSize`. So the same failure is reachable with parameters that pass factory checks—this is the deployment-side root cause of the runtime revert.

1. Factory: no coupling between seed economics and ladder sizing

```
function _validateOrderBookConfig(
    IMarket.OrderBookSeedConfig memory orderBookSeedConfig
) internal pure {
    if (orderBookSeedConfig.minOrderSize == 0) {
        revert IMarketFactory.InvalidOrderBookConfig();
    }
    if (
        orderBookSeedConfig.liquidityGap == 0
        || orderBookSeedConfig.orderAmount == 0
    ) revert IMarketFactory.InvalidOrderBookConfig();
    if (
        orderBookSeedConfig.minTick == 0
        || orderBookSeedConfig.maxTick < orderBookSeedConfig.minTick
    ) revert IMarketFactory.InvalidTickBounds();
    if (orderBookSeedConfig.minCoverage == 0) {
        revert IMarketFactory.InvalidMinCoverage();
    }

    uint256 ladderCoverage = (orderBookSeedConfig.orderAmount >> 1)
        * orderBookSeedConfig.liquidityGap;
    if (ladderCoverage < orderBookSeedConfig.minCoverage) {
        revert IMarketFactory.InsufficientLadderCoverage();
    }
}
```

Nothing here lower-bounds `seedFund` at graduation against `orderAmount`, `minOrderSize`, and outcome count `n`.

2. Ask ladder: proportional share then `minOrderSize` check

```
uint256 allocated = 0;
for (uint256 k; k < validLevels;) {
    prices[k] = impliedPriceBps + (k + 1) * gap;

    if (k < validLevels - 1) {
        amounts[k] = (askTokens * (lSquared - k * k)) / wSum;
    } else {
        amounts[k] = askTokens - allocated;
    }
    if (amounts[k] < minOrderSize_) {
        revert IOrderBook.InsufficientSeedFund();
    }
}
```

The last level is the residual `askTokens - allocated`; it can still be `< minOrderSize_` when the taper leaves too little per level for the given `askTokens`.

3. Bid ladder: same gate (often tighter when implied price is low)

```
uint256 colAtLevel;
if (k < validLevels - 1) {
    colAtLevel = (bidCollateral * (lSquared - k * k)) / wSum;
    allocdCol += colAtLevel;
} else {
    colAtLevel = bidCollateral - allocdCol;
}

amounts[k] = (colAtLevel * BPS_DENOMINATOR) / prices[k];
if (amounts[k] < minOrderSize_) {
    revert IOrderBook.InsufficientSeedFund();
}
```

4. Market: if `seedFund > 0`, seeding must succeed or `graduate()` reverts

```
function _seedOrderBookLiquidity(
    address triggeredBy,
    uint256 totalPoolAtGraduation,
    uint256 vaultAssetsRedeemed
) internal {
    uint256 seedFundValue = seedFund;
    if (seedFundValue == 0) {
        emit MarketGraduated(
            triggeredBy, totalPoolAtGraduation, 0, vaultAssetsRedeemed
        );
        return;
    }

    uint256 n = pools.length;
    uint256 totalPoolValue = totalPool;
    IMarket.OrderBookSeedConfig memory cfg = orderBookSeedConfig;
```

```
uint256 levelsPerSide = cfg.orderAmount / 2;
if (levelsPerSide == 0) return;

uint256 bidFund = seedFundValue / 2;
uint256 askFund = seedFundValue - bidFund;
```

With `seedFundValue > 0`, `buildSeedLadders / seedLiquidity` runs; `InsufficientSeedFund` reverts the entire `graduate()` (state including `graduated = true` from that call is rolled back).

5. Numeric sketch (ask side)

With `n = 3`, `minOrderSize = 1_000_000`, `askFund / 3 ≈ 83_333_333`, `levelsPerSide = 25`, `wSum = 10625`, last weight `49`:

`floor(83_333_333 * 49 / 10625) = 384_313 < 1_000_000 → revert.`

A separate “factory-only” style example: modest `graduationThreshold` and seed fee with large `minOrderSize` and `orderAmount` can pass `_validateOrderBookConfig` while implied `seedFund` at threshold stays far below what $\Theta(L^2) \times \text{minOrderSize} \times n$ requires.

Impact

Availability / product: `totalPool` can reach `graduationThreshold` while `graduate()` still always reverts once `seedFund > 0`, so Phase 2 never appears. **Funds:** Not a direct theft path from this revert; exit still depends on resolution/cancel flows. **Attacker model:** Primarily misconfiguration by a privileged deployer; not a typical unprivileged exploit.

Recommendations

- Factory (prevention):** At market creation, conservatively estimate `seedFund` at `graduationThreshold` from fee tiers and require it $\geq$ a lower bound derived from worst-case tapering over `L`, `minOrderSize`, and `n` (both sides, worst implied prices), or cap `orderAmount` given the rest. Revert with a dedicated error when the combo is impossible.
- Runtime (containment):** If product prefers liveness over strict ladder shape, reduce `validLevels` or skip levels below `minOrderSize` and redistribute instead of reverting.

Status

Fixed — melanke: I noticed this problem when I built a UI prototype to get a feel for the market end-to-end, and ended up adding a front-end simulation in the create-market form that mirrors the seeding math (estimates `seedFund` at graduation, runs both ladders per outcome, surfaces `InsufficientSeedFund` reasons before broadcasting). Re-reading the issue, here's how I'm thinking about the options.

[M-05] Zero vault redemption snapshot breaks idempotency on the second `_finalizeVaultAccounting` call

Description

`_finalizeVaultAccounting` skips re-redemption when `resolvedVaultAssetsSnapshot > 0`, treating a positive snapshot as “already finalized.” When the vault returns 0 assets, the snapshot is stored as 0, so the guard does not fire on a later call. `totalVaultSharesTracked` is never decremented after `vault.redeem`, so the second call attempts to redeem the same share amount again; typical ERC4626-style vaults revert (insufficient shares). `graduate()` and `_finalizeResolvedMarket()` both call `_finalizeVaultAccounting`, so a graduated market that redeemed to zero at graduation cannot complete resolution finalization. This is not NM-001 (vault `redeem` reverting on the first call): the first redeem succeeds with zero assets; the bug is the Market’s reuse of `snapshot > 0` as the only “done” signal.

1. Idempotency tied to a strictly positive snapshot

```
function _finalizeVaultAccounting()
    internal
    returns (uint256 redeemedAssets)
{
    if (resolvedVaultAssetsSnapshot > 0) {
        return resolvedVaultAssetsSnapshot;
    }
    if (address(vault) == address(0) || totalVaultSharesTracked == 0) {
        return 0;
    }

    redeemedAssets =
        vault.redeem(totalVaultSharesTracked, address(this), address(this));
    _applyVaultLossWaterfall(redeemedAssets);
    resolvedVaultAssetsSnapshot = redeemedAssets;

    emit VaultAccountingFinalized(totalVaultSharesTracked, redeemedAssets);
}
```

If `redeemedAssets == 0`, the next entry still has `resolvedVaultAssetsSnapshot == 0` and `totalVaultSharesTracked > 0`, so it enters `vault.redeem` again.

2. Graduation already calls vault finalization once

```
uint256 vaultAssetsRedeemed = _finalizeVaultAccounting();
totalPoolValue = totalPool;
```

3. Resolution calls it again inside `_finalizeResolvedMarket`

```
resolved = true;
actualResolutionTime = block.timestamp;
_finalizeVaultAccounting();
```

After graduation with total vault loss: first call burned shares and set `snapshot = 0`; second call re-enters `vault.redeem(totalVaultSharesTracked, ...)` and reverts.

4. Non-graduated markets

`cancel()` / `resolve()` paths that finalize without an earlier graduation only invoke `_finalizeVaultAccounting` once in `_finalizeResolvedMarket` or `_finalizeCancelledMarket`, so the double-entry path does not apply the same way.

Impact

Liveness: Graduated market may be unable to finish `processFinalizationBatch` → `_finalizeResolvedMarket` if the vault fully lost assets before or at graduation redemption. **Funds:** Post-graduation CLOB collateral and claims can be stuck behind a permanently reverting finalization.

- The likelihood of this is low (total loss + graduated path), but the severity of stuck state is high enough to have us fix this.

Recommendations

Track completion with a dedicated boolean (`vaultAccountingFinalized`) set after the first successful redeem attempt, and return `resolvedVaultAssetsSnapshot` on subsequent calls regardless of whether it is zero. Optionally zero `totalVaultSharesTracked` after redeem if the rest of the system allows, but the boolean guard is the minimal fix.

Status

Fixed

[M-06] Cancel without refund sends only `totalPool + seedFund`, not full redeemed vault balance

Description

`_finalizeCancelledMarket` always calls `_finalizeVaultAccounting()` first, so the Market holds the actual ERC20 proceeds of redeeming all tracked vault shares (`redeemedAssets`). On the no-refund branch, it transfers `totalPool + seedFund` to the treasury and zeros those trackers. If the vault outperformed accounting (`redeemedAssets > totalPool + seedFund`), the excess (vault yield on top of what was modeled as pool + seed) stays on the Market contract. `claimYield` is unavailable (`!resolved`). Users on cancel-with-refund have a different gap (principal-only refunds). Here the intended recipient of everything—the treasury—still does not receive the yield leg without a separate sweep.

1. Cancel finalization: redeem then optional treasury transfer

```
function _finalizeCancelledMarket(bool refund) internal {
    cancelled = true;
    actualResolutionTime = block.timestamp;
    _finalizeVaultAccounting();

    if (refund) {
        uint256 poolCount = pools.length;
        cancelledOutcomeSupplyAtCancellation = new uint256[](poolCount);
        for (uint256 i = 0; i < poolCount; i++) {
            // exclude outcome tokens currently held by this contract from the refund denominator.
            uint256 internalOutcomeBalance =
                OutcomeToken(outcomeToken).balanceOf(address(this), i);
            cancelledOutcomeSupplyAtCancellation[i] = OutcomeToken(
                outcomeToken
            ).totalSupply(i) - internalOutcomeBalance;
        }
    }
}
```

```

    }
  } else {
    uint256 collateralToTreasury = totalPool + seedFund;
    if (collateralToTreasury > 0) {
      collateralToken.safeTransfer(treasury, collateralToTreasury);
      totalPool = 0;
      seedFund = 0;
    }
  }
}

emit MarketCancelled(refund);
}

```

There is no branch that sends `collateralToken.balanceOf(address(this))` or `resolvedVaultAssetsSnapshot` minus fees on cancel-no-refund.

2. When `_applyVaultLossWaterfall` does not absorb the gap

If `redeemedAssets >= totalPool + seedFund`, the waterfall returns without reducing pools; `totalPool + seedFund` still reflects pre-redeem book totals, not the full physical balance after redeem.

```

function _applyVaultLossWaterfall(uint256 redeemedAssets) internal {
  uint256 expectedAssets = totalPool + seedFund;
  if (redeemedAssets >= expectedAssets) return;
}

```

So any vault surplus above `expectedAssets` remains as ERC20 on the Market after the treasury transfer of `totalPool + seedFund`.

3. Recovery is not automatic for collateral

Factory-only `scheduleEmergencyWithdraw` / `executeEmergencyWithdraw` on `collateralToken` is gated on resolution/cancel state and `actualResolutionTime + COLLATERAL_CLAIM_PERIOD` (90 days) for collateral—so stranded yield is not immediately treasury-accessible.

```

function _validateCollateralEmergencyWithdraw(address token) internal view {
  if (token != address(collateralToken)) return;
  if (!resolved && !cancelled) revert IMarket.MarketStillActive();
  if (actualResolutionTime == 0) revert IMarket.MarketNotResolved();
  if (block.timestamp <= actualResolutionTime + COLLATERAL_CLAIM_PERIOD) {
    revert IMarket.ClaimPeriodNotEnded();
  }
}

```

Impact

Treasury: Intended “sweep all value” on no-refund cancel misses vault upside. **Ops:** Stranded balance until timelocked emergency path (and only if factory schedules it). **Scale:** Grows with vault APY and time-in-vault.

Recommendations

On cancel-no-refund, transfer the full collateral balance after vault redeem (or `max(tracked, resolvedVaultAssetsSnapshot)` minus explicit in-flight escrows if any), then zero accounting. Alternatively transfer `collateralToken.balanceOf(address(this))` after resetting internal liabilities, with clear ordering so user refunds (refund path) are not double-hit.

Status

Fixed

[M-07] Vault redeem revert can permanently block market lifecycle finalization

Description

`_finalizeVaultAccounting()` makes a bare external call to `vault.redeem(...)` with no fallback path. That function is on the critical path for `graduate`, `resolve` (directly or via finalization batch), and `cancel`. If redeem reverts, all those transitions revert as well. Because collateral emergency withdrawal for the market token is gated behind finalized state (`resolved || cancelled`) and claim-window delay, operations cannot use emergency withdrawal to break the deadlock while the market is still active.

1. Bare external redeem in `_finalizeVaultAccounting`

```
function _finalizeVaultAccounting()
    internal
    returns (uint256 redeemedAssets)
{
    if (resolvedVaultAssetsSnapshot > 0) {
        return resolvedVaultAssetsSnapshot;
    }
    if (address(vault) == address(0) || totalVaultSharesTracked == 0) {
        return 0;
    }

    redeemedAssets =
        vault.redeem(totalVaultSharesTracked, address(this), address(this));
    _applyVaultLossWaterfall(redeemedAssets);
    resolvedVaultAssetsSnapshot = redeemedAssets;

    emit VaultAccountingFinalized(totalVaultSharesTracked, redeemedAssets);
}
```

No `try/catch` or alternate settlement path exists here.

2. Lifecycle functions that depend on this path

```
function graduate() external nonReentrant onlyNotGraduated {
    if (resolved || cancelled) revert IMarket.AlreadyFinalized();
    if (paused) revert IMarket.MarketNotOpen();
    // ...
    graduated = true;
    _recordActivityTimestamp();

    uint256 vaultAssetsRedeemed = _finalizeVaultAccounting();
}
```

```
function resolve(uint8 correctOutcome_) external nonReentrant onlyFactory {
    // ...
}
```

```

    if (graduated) {
        _startPendingFinalization(
            IMarket.PendingFinalizationType.RESOLVE, correctOutcome_, false
        );
        return;
    }

    _finalizeResolvedMarket(correctOutcome_);
}

```

```

function _finalizeCancelledMarket(bool refund) internal {
    cancelled = true;
    actualResolutionTime = block.timestamp;
    _finalizeVaultAccounting();
}

```

```

IMarket.PendingFinalizationType finalizationType = pendingFinalizationType;
// ...
if (finalizationType == IMarket.PendingFinalizationType.RESOLVE) {
    _finalizeResolvedMarket(correctOutcome_);
} else {
    _finalizeCancelledMarket(refund);
}

```

Any revert in `vault.redeem` bubbles up and blocks finalization.

3. Emergency withdrawal does not unlock this active-state deadlock

```

if (token_ != address(0)) {
    _validateCollateralEmergencyWithdraw(token_);
}

uint256 readyAt = block.timestamp + emergencyWithdrawDelay;
pendingEmergencyWithdraw = IMarket.PendingEmergencyWithdraw({
    token: token_, to: to_, amount: amount_, executeAfter: readyAt
});

```

```

function _validateCollateralEmergencyWithdraw(address token) internal view {
    if (token != address(collateralToken)) return;
    if (!resolved && !cancelled) revert IMarket.MarketStillActive();
    if (actualResolutionTime == 0) revert IMarket.MarketNotResolved();
    if (block.timestamp <= actualResolutionTime + COLLATERAL_CLAIM_PERIOD) {
        revert IMarket.ClaimPeriodNotEnded();
    }
}

```

If lifecycle cannot reach `resolved` / `cancelled`, collateral emergency withdrawal cannot be scheduled/executed for the market token.

Impact

- A vault redeem failure (temporary pause, liquidity crunch, vault bug, or malicious behavior) can freeze graduation/resolution/cancellation progress for that market. Users are stuck waiting for external vault recovery or protocol intervention outside normal flows.

Recommendations

Add failure-tolerant finalization logic in `_finalizeVaultAccounting`, e.g. `try/catch` with partial redeem (`maxRedeem`) or an explicit degraded mode that snapshots redeemable assets and proceeds with waterfall on available funds. Also consider an explicit factory emergency override for irrecoverable vault failures.

Status

Fixed

[M-08] `buyOnPhase1` slippage check protects the wrong direction for buyers

Description

`buyOnPhase1` checks `impliedOddsBps < minAcceptableOdds_` and reverts in that case. But buyer output is computed as `tokensOut = collateral * 10_000 / effectivePriceBps`, where `effectivePriceBps` increases with `impliedOddsBps`. So higher implied odds means fewer tokens out (worse execution). The current check blocks only lower odds (better execution), not higher odds (worse execution). This leaves users exposed to adverse price movement between signing and execution.

1. Current guard in `buyOnPhase1`

```
function buyOnPhase1(
    uint8 outcomeIndex_,
    uint256 collateralAmount_,
    uint256 minAcceptableOdds_
) external nonReentrant onlyOpen onlyNotGraduated {
    // ...
    // Slippage protection: check if current odds meet minimum requirement
    uint256 impliedOddsBps = _getBuyImpliedOdds(outcomeIndex_);
    if (minAcceptableOdds_ > 0 && impliedOddsBps < minAcceptableOdds_) {
        revert IMarket.SlippageTooHigh();
    }
}
```

This is a lower-bound check on odds.

2. Execution math: higher implied odds means fewer tokens

```
uint16 seedFeeBps = _getSeedFeeBpsByPoolSize(totalPool);
uint256 effectivePriceBps =
    (impliedOddsBps * (BPS_DENOMINATOR + seedFeeBps)) / BPS_DENOMINATOR;
if (effectivePriceBps == 0) revert IMarket.ZeroTokenOutput();

uint256 tokensOut =
    (collateralAmount_ * BPS_DENOMINATOR) / effectivePriceBps;
if (tokensOut == 0) revert IMarket.ZeroTokenOutput();
```

Because `tokensOut` is inverse to `effectivePriceBps`, adverse movement for a buyer is upward odds/price, not downward.

3. Why this is exploitable in practice

A user can submit with `minAcceptableOdds_` expecting it to cap bad execution. If the market moves to higher implied odds before inclusion, the tx still passes this check and executes with fewer tokens out. Lower implied odds (better fill) would have been blocked instead.

Impact

- Buyers cannot express a true “max price / worst acceptable fill” bound in this function. They remain exposed to adverse execution from mempool/sequencer ordering and normal price drift.

Recommendations

Replace the guard with one of:

- `maxAcceptableOdds_` and revert when `impliedOddsBps > maxAcceptableOdds_`, or
- `minTokensOut` and revert when computed `tokensOut < minTokensOut` (preferred standard).

Status

Fixed

[M-09] `marketBuy` slippage guard excludes taker fee, allowing total spend to exceed `maxCollateralIn`

Description

`marketBuy` in `CLOBRBTTree.sol` accepts `maxCollateralIn` as the caller’s spending cap. The NatSpec describes it as “Maximum collateral allowed to be spent.” But the slippage check only compares the raw fill notional against this cap - it excludes the taker fee:

```
if (spentCollateral > maxCollateralIn) revert SlippageExceeded();

uint256 spentWithFees = spentCollateral + paidTakerFee; // <- actual deduction
uint256 refund = (maxNotional + maxTakerFee) - spentWithFees;
```

The actual amount deducted from the caller is `spentCollateral + paidTakerFee`, which can exceed `maxCollateralIn` by the full taker fee amount.

The sell-side is inconsistent: `marketSell` checks `receivedCollateral < minCollateralOut` where `receivedCollateral` is already net of taker fees. So sells guard post-fee, buys guard pre-fee.

Impact

- With a 1% taker fee and `maxCollateralIn = 10,000 USDC`, a full fill deducts 10,100 USDC while the guard passes at exactly 10,000. Users treating this parameter as a budget cap silently overspend by the fee amount on every market buy. The overspend is unrecoverable.

Recommendations

Check against the fee-inclusive total, matching the sell-side’s post-fee semantics in `CLOBRBTTree.sol`:

```
uint256 spentWithFees = spentCollateral + paidTakerFee;
if (spentWithFees > maxCollateralIn) revert SlippageExceeded();
uint256 refund = (maxNotional + maxTakerFee) - spentWithFees;
```

Status

Fixed

[M-10] Seed ask-token flow increases winning-token denominator without adding to `payoutPool`

Description

During graduation seeding, Market mints ask-side outcome tokens from `askFund` and places them on CLOB, then sets `seedFund = 0` without adding ask backing to `totalPool/pools[]`. If those seed asks fill, tokens move to traders and are counted in `OutcomeToken.totalSupply()` at resolution, but associated collateral flows are not incorporated into `payoutPool` (which is based on `totalPool`). This can dilute per-token winner payout and leave collateral balances outside primary payout accounting.

1. Ask-side seeding mints tokens from seed fund, then zeroes `seedFund`

```
uint256 bidFund = seedFundValue / 2;
uint256 askFund = seedFundValue - bidFund;
// ...
uint256 askTokens = (i == n - 1) ? askBase + askRem : askBase;
if (askTokens > 0) {
    OutcomeToken(outcomeToken).mint(address(this), i, askTokens);
}
_seedOutcomeBook(
    i, impliedPriceBps, bidAlloc, askTokens, levelsPerSide, cfg
);
// ...
seedFund = 0;
```

These minted ask tokens are inventory for seed sell orders.

2. Resolution payout pool uses `totalPool`, while denominator uses token supply

```
uint256 internalWinningBalance = OutcomeToken(outcomeToken)
    .balanceOf(address(this), correctOutcome_);
winningTokenSupplyAtResolution = OutcomeToken(outcomeToken)
    .totalSupply(correctOutcome_) - internalWinningBalance;
```

```
// ...
uint256 totalPoolValue = totalPool;
uint256 correctPool = pools[correctOutcome_];
// ...
payoutPool = correctPool == 0 ? 0 : totalPoolValue - rake;
```

So filled seed-ask tokens held by users increase denominator, while payout numerator is not seed-flow-aware.

3. Seed order flows are settled via CLOB, not pool accounting

```
if (totalBidNotional > 0) {
    collateralToken.safeTransfer(clobAddr, totalBidNotional);
}
if (totalAskTokens > 0) {
    if (!OutcomeToken(outcomeToken)
        .transfer(clobAddr, outcomeIdx, totalAskTokens)) {
        revert IMarket.TransferFailed();
    }
}

IOrderBook(clobAddr)
    .seedLiquidity(bidPrices, bidAmounts, askPrices, askAmounts);
```

There is no callback path that reconciles post-seed CLOB settlement proceeds into `totalPool/pools[]`.

Impact

- Economic mismatch between winner-token supply and payout numerator can reduce payout per winning token relative to intuitive “fully backed” expectations. Some seed-related collateral can remain outside explicit payout allocation, recoverable only through later administrative sweep paths.

Recommendations

Define and enforce one accounting policy:

- either fold seed-related realized collateral into payout accounting at resolution, or
- explicitly route/sweep seed residuals (e.g., treasury) and document user-facing payout semantics.

Also consider recording explicit seed-accounting state during Phase 2 to avoid implicit balance-based leftovers.

Status

Fixed

Low

[L-02] Complete-set mint when `amount` is smaller than outcome count

Description

`mintCompleteSet(amount)` is supposed to read like “deposit `amount` collateral and get a full set of outcome tokens.” Under the hood it splits `amount` across `n` outcomes using integer division: `base = amount / n`, remainder dumped on the last outcome index.

When `amount < n` (in raw token units, wei / USDC base units), `base` is zero. The mint loop skips every outcome with zero allocation, so only the last outcome gets minted — you’ve effectively bought a single leg at 1:1 token-to-collateral on that leg, not a “complete set” in the everyday sense.

The protocol still balances: `totalPool` goes up by the full `amount`, and the sum of `pools[i]` deltas equals `amount`. Nobody’s stealing from the pool; the issue is surprising behavior and easy-to-misuse API if a frontend passes tiny amounts or assumes every outcome always gets non-zero tokens.

1. How the split works

```
function _getCompleteSetBaseAndRemainder(uint256 totalAmount)
    internal
    view
    returns (uint256 baseAmount, uint256 remainder)
{
    uint256 n = pools.length;
    baseAmount = totalAmount / n;
    remainder = totalAmount % n;
}
```

If `totalAmount < n`, then `baseAmount == 0` and `remainder == totalAmount`.

2. What the mint loop actually does

```
for (uint256 i = 0; i < outcomesLength; i++) {
    uint256 outcomeAmount =
        i == lastOutcomeIndex ? baseAmount + remainder : baseAmount;
    if (outcomeAmount == 0) continue;
    pools[i] += outcomeAmount;
    OutcomeToken(outcomeToken).mint(msg.sender, i, outcomeAmount);
}
totalPool += amount_;
```

So for `n = 6` and `amount_ = 5`: outcomes `0...4` hit `continue`; outcome `5` gets `5` units and `pools[5] += 5`. You paid 5 units of collateral for 5 units of one outcome’s token supply, not one token on each leg.

For normal sizes (e.g. 1 USDC with 6 decimals and 6 outcomes), `base` is non-zero and you get a real spread plus remainder on the last index — the edge case is specifically `amount_ < n`.

Quick numbers

n	amount_	base	remainder	Who gets tokens?
6	5	0	5	Last outcome only, quantity 5
6	1_000_000	166_666	4	All outcomes; last gets +4 dust

`minTradeSize` applies to Phase 1 `buyOnPhase1`, not to `mintCompleteSet`, so nothing stops a caller from using `amount_ = 1` if they want.

Impact

- Mis-integrated UIs that call `mintCompleteSet(1)` (or any `amount < n`) thinking it's "one tick of a full set" will mint **only the last outcome**. That can be confusing or economically silly depending on pool state, but it's not the contract double-charging or breaking `sum(pools) == totalPool`.
- No insolvency path from this, the invariant that matters for collateral vs pool state still lines up.

Recommendations

Pick one (I'd do the first if you want minimal behavior change):

1. **Revert small amounts:** `require(amount_ >= pools.length, ...)` so you never silently mint a single leg from a "complete set" entrypoint.
2. **Stricter symmetry:** `require(amount_ % pools.length == 0)` if you want every leg to get exactly the same increment.
3. **Docs only:** If you keep the math as-is, NatSpec on `mintCompleteSet` should spell out that remainder always sits on the last outcome and that `amount < n` means only that outcome receives tokens.

Status

Fixed

[L-03] `cancelAllOrders` resets the resolution-delay clock

Description

`resolve` only runs if `block.timestamp` is at least `resolutionDelay` seconds after the latest activity timestamp the contract knows about. That "latest" value is the maximum of `phase1LastActivityTimestamp` on the Market and `lastActivityTimestamp` on each CLOB.

`cancelAllOrders` is implemented as: clear every book, then refresh those clocks. The Market calls `_recordActivityTimestamp()` (Phase 1 → `block.timestamp`). Each CLOB's `cancelOrdersRange` loop ends with `_recordActivityTimestamp()` on the book (`block.timestamp` again). So after a successful cancel-all, the delay gate measures from the cancel-all transaction, not from "the last trade before we cleaned up."

If its assumed that "empty book ⇒ we can resolve now," the next `resolve` will `MustWaitAfterLastTrade` until another full `resolutionDelay` has elapsed from cancel-all. The books really are empty, the timer was just pushed forward by the cleanup path itself.

1. `cancelAllOrders` on the Market

```
function cancelAllOrders()
  external
  nonReentrant
  onlyFactory
  onlyGraduated
  onlyNotFinalizing
{
  _cancelAllOutcomeOrderBooks();
  _recordActivityTimestamp();
}
```

```
function _recordActivityTimestamp() internal {
  phase1LastActivityTimestamp = block.timestamp;
}
```

2. End of each CLOB `cancelOrdersRange` pass

`cancelAllOrders` on the CLOB repeatedly runs `_cancelOrdersRange`; every pass ends with:

```
_recordActivityTimestamp();
return (orderId, orderId >= finalOrderId);
```

So each book's `lastActivityTimestamp` moves to the time of that cancellation work.

3. What `resolve` checks

```
uint256 latestActivityTimestamp = _getLatestActivityTimestamp();
if (latestActivityTimestamp > 0 && timing.resolutionDelay > 0) {
  if (
    block.timestamp
    < latestActivityTimestamp + timing.resolutionDelay
  ) revert IMarket.MustWaitAfterLastTrade();
}
```

4. How `latestActivityTimestamp` is built

```
function _getLatestActivityTimestamp()
  internal
  view
  returns (uint256 latestTimestamp)
{
  latestTimestamp = phase1LastActivityTimestamp;

  uint256 orderBooksLength = outcomeOrderBooks.length;
  for (uint256 i = 0; i < orderBooksLength; i++) {
    uint256 orderBookActivityTimestamp =
      IOrderBook(outcomeOrderBooks[i]).lastActivityTimestamp();
    if (orderBookActivityTimestamp > latestTimestamp) {
      latestTimestamp = orderBookActivityTimestamp;
    }
  }
}
```

```

    }
  }
}

```

Immediately after cancel-all, that max is tied to now (the cancel pass). `resolve` therefore needs `now + resolutionDelay`, not “delay since last fill.”

Impact

- Failed `resolve` calls, repeated retries, wasted gas. No direct fund loss from this mechanism.
- Mass cancel reads as housekeeping. the delay logic treats it the same as any other activity bump. If the market was still open for new orders, traders could also keep moving the clock; cancel-all on top **adds** another fresh baseline rather than shortening the wait.

Recommendations

1. For factory-driven `cancelAllOrders`, consider not calling `_recordActivityTimestamp` on the Market and not bumping CLOB timestamps on the market-initiated cancel-range path (or gate it behind a flag), if product intent is “admin cleanup doesn’t restart the resolution timer.”
2. Document that `resolutionDelay` is measured from cancel-all if that runs before `resolve`.

Status

Fixed — **melanke**: That’s fair: in practice it can be inconvenient for ops who expect “book is empty → we can resolve immediately,”.

[L-05] `levelsPerSide * levelsPerSide` can overflow before ladder logic runs

Description

`_computeBidLadder` and `_computeAskLadder` set `lsquared = levelsPerSide * levelsPerSide` with no guard. In Solidity 0.8.x, `uint256` multiplication overflows revert. `levelsPerSide` is `orderBookSeedConfig.orderAmount / 2` at seed time. The factory enforces `orderAmount != 0` and a minimum ladder coverage product `(orderAmount >> 1) * liquidityGap >= minCoverage`, but does not cap `orderAmount`. In principle one can choose `orderAmount` so large that $(\text{orderAmount}/2)^2$ exceeds $2^{256}-1$; factory checks can still pass with small `liquidityGap` and `minCoverage` (gap `1`, minCoverage `1`). Then every `graduate()` that reaches seeding with `seedFund > 0` hits this revert.

1. Unchecked square in both ladder builders

```
uint256 lsquared = levelsPerSide * levelsPerSide;
```

```
uint256 lsquared = levelsPerSide * levelsPerSide;
```

2. Factory: coverage only, no upper bound on `orderAmount`

```
function _validateOrderBookConfig(
    IMarket.OrderBookSeedConfig memory orderBookSeedConfig
) internal pure {
    if (orderBookSeedConfig.minOrderSize == 0) {
        revert IMarketFactory.InvalidOrderBookConfig();
    }
    if (
        orderBookSeedConfig.liquidityGap == 0
        || orderBookSeedConfig.orderAmount == 0
    ) revert IMarketFactory.InvalidOrderBookConfig();
    // ...
    uint256 ladderCoverage = (orderBookSeedConfig.orderAmount >> 1)
        * orderBookSeedConfig.liquidityGap;
    if (ladderCoverage < orderBookSeedConfig.minCoverage) {
        revert IMarketFactory.InsufficientLadderCoverage();
    }
}
```

3. Orders of magnitude

Overflow requires `levelsPerSide` $\geq 2^{128}$ (since $(2^{128})^2 = 2^{256}$). Such a loop could not run on-chain in practice; the overflow revert is reached before the ladder loops iterate that far. So this is a configuration brick, not a practical gas grief from iterating `2128` levels.

Impact

- market may never graduate if an enormous `orderAmount` is ever allowed. Only a privileged / mistaken factory configuration. End-user funds are not targeted by the overflow itself.

Recommendations

In `_validateOrderBookConfig` (or in `CLOB` as defense in depth), reject `orderAmount >> 1` above a small constant derived from gas limits (hundreds or low thousands of levels), well below `2128`.

Status

Fixed

[L-06] Empty bid ladder leaves that outcome's `bidAlloc` collateral in the Market after `seedFund` is cleared

Description

`_computeBidLadder` may return zero valid levels when the first gap step already violates the bid price window (`offset >= impliedPriceBps` or `impliedPriceBps - offset < minTick`). That happens, for example, when `impliedPriceBps` is forced to `1` (from a zero pool share) and `liquidityGap` ≥ 1 . Market still computed `bidAlloc` for that outcome from `bidFund`; if the ladder is empty, `totalBidNotional` is zero and no collateral is transferred to the CLOB for bids. After the seeding loop, `seedFund` is set to `0` while any unsent bid collateral remains as ERC20 balance on the Market, its not tracked in `seedFund`. The same path can combine with double-rounding dust, amounts are tiny when the outcome's pool share is tiny.

1. Bid ladder: immediate break for extreme implied price

```
for (uint256 k; k < levelsPerSide;) {
    uint256 offset = (k + 1) * gap;
    if (
        offset >= impliedPriceBps || impliedPriceBps - offset < minTick_
    ) break;
    wSum += lSquared - k * k;
    unchecked {
        ++validLevels;
        ++k;
    }
}
if (validLevels == 0) return (new uint256[])(0), new uint256[](0), 0);
```

2. Market: `impliedPriceBps` floor to 1; `bidAlloc` still computed

```
for (uint256 i; i < n;) {
    uint256 impliedPriceBps =
        (pools[i] * BPS_DENOMINATOR) / totalPoolValue;
    if (impliedPriceBps == 0) impliedPriceBps = 1;

    uint256 bidAlloc = (i == n - 1)
        ? bidFund - allocSum
        : Math.mulDiv(bidFund, pools[i], totalPoolValue);
    if (i < n - 1) {
        allocSum += bidAlloc;
    }
    // ...
    _seedOutcomeBook(
        i, impliedPriceBps, bidAlloc, askTokens, levelsPerSide, cfg
    );
};
```

3. Outcome book: transfer only `totalBidNotional`

```
if (bidPrices.length == 0 && askPrices.length == 0) return;

if (totalBidNotional > 0) {
    collateralToken.safeTransfer(clobAddr, totalBidNotional);
}
```

If the ask side produced orders, seeding continues; bid collateral for that outcome is not sent when the bid ladder is empty.

Impact

Accounting: Small residual collateral on the Market contract, not reflected in `seedFund` after graduation seeding.

Economic: Bounded by outcomes with near-zero weight in `bidFund`, typically sub-min-notional dust. **Recovery:** Only indirect paths (long-horizon admin/treasury flows if any exist for stray balances), not normal user withdraw math.

Recommendations

After seeding, sweep `collateralToken.balanceOf(address(this))` minus explicitly tracked liabilities (or the delta between `bidFund` and sum of `totalBidNotional` actually sent) to `treasury`, or add a dedicated `seedFund` decrement that matches sent amounts only. Alternatively document the dust as acceptable. Prefer explicit transfer of “unused bid

allocation” when `bidPrices.length == 0`.

Status

Fixed

[L-07] `graduate()` can execute after market close while CLOB placement is already closed

Description

`graduate()` does not use `onlyOpen` and therefore does not enforce `closed` / `closeTime` / `resolutionTime` open-window checks. As a result, graduation can proceed after trading closure conditions. But CLOB order placement checks those same closure conditions and rejects new order placement. This can create a graduated market with seeded books but effectively closed trading surface.

1. `graduate()` lacks open-window checks

```
function graduate() external nonReentrant onlyNotGraduated {
    if (resolved || cancelled) revert IMarket.AlreadyFinalized();
    if (paused) revert IMarket.MarketNotOpen();

    uint256 totalPoolValue = totalPool;
    bool isFactoryTrigger = msg.sender == factory;
    if (!isFactoryTrigger && totalPoolValue < graduationThreshold) {
        revert IMarket.GraduationThresholdNotMet();
    }
}
```

No `closed` or timing gate appears here.

2. Normal trading paths use `onlyOpen`

```
modifier onlyOpen() {
    _onlyOpen();
    _;
}
```

```
function _onlyOpen() internal view {
    if (closed || resolved || cancelled) revert IMarket.MarketNotOpen();
    if (paused) revert IMarket.MarketNotOpen();
    if (block.timestamp < timing.openTime) {
        revert IMarket.MarketNotOpen();
    }
    if (timing.closeTime != 0 && block.timestamp >= timing.closeTime) {
        revert IMarket.MarketNotOpen();
    }
    if (
        timing.closeTime == 0 && timing.resolutionTime != 0
        && block.timestamp >= timing.resolutionTime
    ) {
        revert IMarket.MarketNotOpen();
    }
}
```

```

    }
}

```

3. CLOB placement enforces closed/timing state

```

function _enforceOrderPlacementStatus() internal view {
    IMarketOrderBookState marketState = IMarketOrderBookState(market);
    uint256 closeTime;
    uint256 resolutionTime;
    (,, closeTime, resolutionTime) = marketState.timing();
    if (!marketState.graduated()) revert IOrderBook.MarketNotGraduated();
    if (marketState.resolved()) revert IOrderBook.MarketResolved();
    if (marketState.cancelled()) revert IOrderBook.MarketCancelled();
    if (marketState.paused()) revert IOrderBook.MarketPaused();
    if (
        marketState.closed()
        || (closeTime != 0 && block.timestamp >= closeTime)
        || (closeTime == 0
            && resolutionTime != 0
            && block.timestamp >= resolutionTime)
    ) revert IOrderBook.MarketClosed();
}

```

So post-close graduation can seed books into a venue that rejects user order entry.

Impact

- Potentially wasted seed-reward/seed-liquidity setup and confusing market state (`graduated` but effectively closed for CLOB trading). Not a direct theft vector.

Recommendations

Align graduation gating with open-window policy (either require open state in `graduate()` or explicitly document/support “post-close graduation for settlement-only” behavior and skip seeding/reward in that mode).

Status

Fixed

[L-08] `calculatePotentialPayout` uses raw token supply instead of excluding market-held tokens

Description

`calculatePotentialPayout` in `Market.sol` computes the payout preview using `OutcomeToken.totalSupply(outcomeIndex)` as the denominator. But actual resolution in `_finalizeResolvedMarket` uses `totalSupply(correctOutcome) - balanceOf(address(this), correctOutcome)`, excluding tokens held by the Market itself (seed ask tokens, returned CLOB orders).

For graduated markets, Market can hold a meaningful amount of seed tokens. The preview denominator is always larger than the real one, so the function systematically underestimates actual payouts. Any frontend or off-chain tool displaying “potential winnings” to users will show lower values than what they’d actually receive on resolution.

Recommendation

Subtract Market's own balance from the denominator in `calculatePotentialPayout`, matching the logic in `_finalizeResolvedMarket`:

```
uint256 internalBalance = OutcomeToken(outcomeToken).balanceOf(address(this), outcomeIndex);
uint256 totalOutcomeTokens = OutcomeToken(outcomeToken).totalSupply(outcomeIndex) - internalBalance;
```

Recommendations

Subtract Market's own balance from the denominator in `calculatePotentialPayout`, matching the logic in `_finalizeResolvedMarket`:

```
uint256 internalBalance = OutcomeToken(outcomeToken).balanceOf(address(this), outcomeIndex);
uint256 totalOutcomeTokens = OutcomeToken(outcomeToken).totalSupply(outcomeIndex) - internalBalance;
```

Status

Fixed — **melanke**: Thanks for catching this, @radeveth — the diagnosis is exactly right.

[L-09] Stale `pools[]` after cancel-without-refund causes misleading view returns

Description

`_finalizeCancelledMarket` with `refund=false` sets `totalPool = 0` and `seedFund = 0` after transferring everything to treasury, but does not zero the individual `pools[]` array entries.

`calculatePayout` is a public view function. After cancel-without-refund, external callers (indexers, analytics dashboards, UIs) calling it for a token-holding user will compute non-zero refund values from stale `pools[i]` data, even though zero collateral backs them. `redeem()` correctly prevents actual claims via its `(totalPool + seedFund) > 0` gate, so no funds are at risk, but a naive frontend could display “You have \$X to claim” when the user actually has nothing.

Recommendation

Zero `pools[]` in the no-refund branch:

```
for (uint256 i = 0; i < pools.length; i++) {
    pools[i] = 0;
}
```

Recommendations

Zero `pools[]` in the no-refund branch:

```
for (uint256 i = 0; i < pools.length; i++) {
    pools[i] = 0;
}
```

Status

Fixed

[L-10] `closeEarly()` leaves `timing.closeTime` stale, breaking `getMarketData()` consistency

Description

`closeEarly()` sets `closed = true` and emits `MarketClosed`, but never updates `timing.closeTime`. After the call, `getMarketData()` returns `closeTime_` pointing to the originally scheduled future time - even though the market is already closed.

```
function closeEarly() external onlyFactory onlyNotFinalizing {
  if (closed || resolved || cancelled) revert AlreadyClosed();
  closed = true;
  emit MarketClosed();
  // timing.closeTime is never updated
}
```

A UI that reads `getMarketData().closeTime_` and uses it to display the close time will show a future timestamp while the market is already closed. The `closed` flag is the source of truth on-chain, but off-chain systems reading the timing struct have no way to distinguish “never reached its closeTime” from “closed early.”

Recommendation

Update `timing.closeTime` to `block.timestamp` when closing early, so the returned timing struct is consistent with the `closed` flag:

```
function closeEarly() external onlyFactory onlyNotFinalizing {
  if (closed || resolved || cancelled) revert AlreadyClosed();
  closed = true;
  timing.closeTime = block.timestamp;
  emit MarketClosed();
  emit TradingWindowUpdated(timing.openTime, block.timestamp);
}
```

Recommendations

Update `timing.closeTime` to `block.timestamp` when closing early, so the returned timing struct is consistent with the `closed` flag:

```
function closeEarly() external onlyFactory onlyNotFinalizing {
  if (closed || resolved || cancelled) revert AlreadyClosed();
  closed = true;
  timing.closeTime = block.timestamp;
  emit MarketClosed();
  emit TradingWindowUpdated(timing.openTime, block.timestamp);
}
```

Status

Fixed

[L-11] `MarketOrderExecuted` event reports pre-fee notional for buys but post-fee proceeds for sells

Description

`MarketOrderExecuted` in both CLOB implementations uses asymmetric fee encoding. For sell-side, `receivedCollateral` is the net post-fee amount the taker actually received. For buy-side, `spentCollateral` is the raw notional excluding taker fees - not the actual amount deducted from the caller.

An off-chain indexer summing `spentCollateral` from buy events will systematically undercount actual collateral outflows by the cumulative taker fees across all market buys. Any fee-inclusive accounting system built on these events will show a collateral deficit.

Recommendation

Emit fee-inclusive spend for buys, matching the sell-side's net semantics in `CLOBRBTtree.sol`:

```
emit MarketOrderExecuted(
    msg.sender, true, amount, receivedTokens, spentCollateral + paidTakerFee, 0
);
```

Recommendations

Emit fee-inclusive spend for buys, matching the sell-side's net semantics in `CLOBRBTtree.sol`:

```
emit MarketOrderExecuted(
    msg.sender, true, amount, receivedTokens, spentCollateral + paidTakerFee, 0
);
```

Status

Fixed

[L-12] Cancel-no-refund zeroes `totalPool` but keeps `pools[]`, causing misleading `calculatePayout()`

Description

On `cancel(refund=false)`, the contract transfers `totalPool + seedFund` to treasury and then sets only those two aggregates to zero. Individual `pools[i]` values are left unchanged. `calculatePayout()` for cancelled markets derives outcome backing from `pools[i]`, so it can return non-zero values even when redemption is impossible (`totalPool + seedFund == 0`). This is mostly a read-path integrity issue (UI/integrator confusion), not direct drain.

1. No-refund cancel branch zeroes aggregates only

```
function _finalizeCancelledMarket(bool refund) internal {
    cancelled = true;
    actualResolutionTime = block.timestamp;
    _finalizeVaultAccounting();

    if (refund) {
        // ...
    } else {
        uint256 collateralToTreasury = totalPool + seedFund;
        if (collateralToTreasury > 0) {
            collateralToken.safeTransfer(treasury, collateralToTreasury);
            totalPool = 0;
            seedFund = 0;
        }
    }
}
```

No reset of `pools[]` in this branch.

2. Cancelled payout view still reads `pools[i]`

```
function calculatePayout(address user_) public view returns (uint256) {
    if (cancelled) {
        uint256 total = 0;
        uint256 totalPoolForSeedSplit = totalPool;
        uint256 seedFundValue = seedFund;
        uint256 poolCount = pools.length;
        for (uint256 i = 0; i < poolCount; i++) {
            // ...
            uint256 outcomeBacking = pools[i];
            if (seedFundValue > 0 && totalPoolForSeedSplit > 0) {
                outcomeBacking += (seedFundValue * pools[i])
                    / totalPoolForSeedSplit;
            }
            total += (userBalance * outcomeBacking) / totalSupply;
        }
        return total;
    }
}
```

If `pools[i]` are stale non-zero while aggregates are zero, the view can overstate claimability.

3. Redeem is still blocked (so this is not direct theft)

```
function _validateRedeemPreconditions() internal view {
    if (resolved) {
        // ...
    } else if (cancelled) {
        if ((totalPool + seedFund) == 0) revert IMarket.NothingToClaim();
    }
}
```

Actual redemption rejects when no refundable backing remains.

Impact

- `calculatePayout()` can report phantom payouts on cancel-no-refund markets, causing UI and integration-level misbehavior (false claimable values, bad off-chain decisions) despite on-chain redeem revert protection.

Recommendations

On no-refund cancellation, also zero `pools[]` (or make `calculatePayout()` short-circuit to 0 when `cancelled && totalPool + seedFund == 0`) so view semantics match redeemability.

Status

Fixed

[L-13] Serial `cancelOrder` calls can delay resolution past intended window

Description

`cancelOrder` in both CLOB implementations is intentionally callable after `closeEarly` so users can recover their escrowed funds. But every call executes `_recordActivityTimestamp()`, which updates the CLOB's `lastActivityTimestamp` to `block.timestamp`. `resolve()` in `Market.sol` checks `_getLatestActivityTimestamp()` - which reads the max across all CLOB timestamps and reverts if `block.timestamp < latestActivity + resolutionDelay`.

After `closeEarly`, no new orders can be placed, but users with resting orders can cancel them one by one. Each cancellation pushes the resolution window forward by up to `resolutionDelay`. A griever with many resting orders spaced across time can meaningfully delay resolution beyond what the admin intended.

The admin has a valid mitigation: calling `cancelAllOrders()` clears every order in one transaction, imposing only a single `resolutionDelay` wait. After that, no orders remain and no further cancellations are possible. The grieving window is bounded by the time between `closeEarly` and the admin's `cancelAllOrders` call.

Impact

- Resolution delayed beyond the intended window if the admin doesn't proactively clear the book. On L2 where gas is cheap, the cost to the griever is minimal per cancellation. and admin can end the grieving at any time with one transaction.

Recommendations

Don't update `lastActivityTimestamp` for cancellations that occur after the market is closed:

```
function cancelOrder(uint256 orderId) external nonReentrant {
    Order storage order = orders[orderId];
    if (order.id == 0) revert OrderNotFound();
    if (!order.active) revert OrderNotActive();
    if (order.trader != msg.sender) revert NotOrderOwner();

    (uint256 refundCollateral, uint256 refundTokens) =
        _cancelActiveOrder(orderId, msg.sender);

    if (!_isMarketClosed()) {
        _recordActivityTimestamp();
    }

    emit OrderCancelled(orderId, msg.sender, refundCollateral, refundTokens);
}
```

Status

Fixed

[L-14] Serial `cancelOrder` calls can delay resolution past intended window

Description

`cancelOrder` in both CLOB implementations is intentionally callable after `closeEarly` so users can recover their escrowed funds. But every call executes `_recordActivityTimestamp()`, which updates the CLOB's `lastActivityTimestamp` to `block.timestamp`. `resolve()` in `Market.sol` checks `_getLatestActivityTimestamp()` - which reads the max across all CLOB timestamps and reverts if `block.timestamp < latestActivity + resolutionDelay`.

After `closeEarly`, no new orders can be placed, but users with resting orders can cancel them one by one. Each cancellation pushes the resolution window forward by up to `resolutionDelay`. A griever with many resting orders spaced across time can meaningfully delay resolution beyond what the admin intended.

The admin has a valid mitigation: calling `cancelAllOrders()` clears every order in one transaction, imposing only a single `resolutionDelay` wait. After that, no orders remain and no further cancellations are possible. The grieving window is bounded by the time between `closeEarly` and the admin's `cancelAllOrders` call.

Impact

- Resolution delayed beyond the intended window if the admin doesn't proactively clear the book. On L2 where gas is cheap, the cost to the griever is minimal per cancellation. and admin can end the grieving at any time with one transaction.

Recommendations

Don't update `lastActivityTimestamp` for cancellations that occur after the market is closed:

```
function cancelOrder(uint256 orderId) external nonReentrant {
    Order storage order = orders[orderId];
    if (order.id == 0) revert OrderNotFound();
    if (!order.active) revert OrderNotActive();
    if (order.trader != msg.sender) revert NotOrderOwner();

    (uint256 refundCollateral, uint256 refundTokens) =
        _cancelActiveOrder(orderId, msg.sender);

    if (!_isMarketClosed()) {
        _recordActivityTimestamp();
    }

    emit OrderCancelled(orderId, msg.sender, refundCollateral, refundTokens);
}
```

Status

Fixed

Informational

[I-01] Append-Only FIFO Queues Let Attackers Inflate Taker Matching Gas

Description

Per price level, resting order IDs are stored in append-only arrays (`buyOrderIdsByPrice[price]` , `sellOrderIdsByPrice[price]`). When a user cancels, the implementation sets `order.active = false` but does not remove the id from the array or compact the queue. The head index (`buyQueueHeadByPrice` / `sellQueueHeadByPrice`) advances only when matching (or when the inner loop skips inactive entries).

An adversary can repeat `place` → `cancel` at the same price many times. That leaves a long prefix of inactive ids before a later active order. A taker whose match walks that price level pays gas for each stale slot: `load queue[head]` , `load orders[id]` , `branch, head++` , `continue`.

This is not a fund theft or accounting issue really, so that's good, the issue is more of an unbounded per-price work for takers, bounded by how much gas the attacker is willing to spend on their own `place/cancel` loop. After one match (or one traversal that advances `head` to the active tail), that pollution at that level is consumed — the design is partially self-healing.

1. Cancellation does not remove queue entries

`_cancelActiveOrder` marks the order inactive and refunds; it does not splice the id out of `sellOrderIdsByPrice` / `buyOrderIdsByPrice`:

```
function _cancelActiveOrder(uint256 orderId, address recipient)
    internal
    returns (uint256 refundCollateral, uint256 refundTokens)
{
    IOrderBook.Order storage order = orders[orderId];
    order.active = false;
    if (order.isBuy) {
        ...
    } else {
        _decrementActiveSellLevel(order.priceBps);
        refundTokens = reservedTokensByOrderId[orderId];
        reservedTokensByOrderId[orderId] = 0;
        if (refundTokens > 0) {
            _safeTransferOutcome(recipient, refundTokens);
        }
    }
}
```

Cancelled ids remain in the dynamic array at their original indices.

2. Matching pays linear cost to skip inactive head entries

```
uint256[] storage queue = sellOrderIdsByPrice[askPrice];
uint256 head = sellQueueHeadByPrice[askPrice];
while (head < queue.length && filledAmount < amount) {
    uint256 makerOrderId = queue[head];
    IOrderBook.Order storage makerOrder = orders[makerOrderId];
    // Lazy cleanup for stale ids.
    if (!makerOrder.active || makerOrder.remaining == 0) {
        head++;
        continue;
    }
}
```

Each `continue` still costs loop work (storage reads, arithmetic). After processing, `head` is written back:

```
sellQueueHeadByPrice[askPrice] = head;
askPrice = _nextActiveSellPrice(askPrice + 1, limitPriceBps);
```

So the first taker that hits a heavily polluted level pays the skip cost; subsequent takers start from the updated `head`.

Impact

- A single trade at a polluted level can cost noticeably more gas or, in an extreme `N`, approach block gas limits (chain-dependent). No incorrect pricing or balance corruption.

Recommendations

1. Document that FIFO queues are append-only and that first match after spam may be gas-heavy.
2. Optional hardening //
 - On cancel, advance `buyQueueHeadByPrice` / `sellQueueHeadByPrice` while the head points at inactive orders (bounded small loop at head only), or
 - Add a permissionless `compactQueue(price, isBuy, maxSteps)` that only moves `head` over inactive entries.

Status

Fixed

[I-02] Transfer lock can be bypassed by burn-style transfer to `address(0)`

Description

`OutcomeToken` blocks transfers while locked only when both `from` and `to` are non-zero. That means any path with `to == address(0)` bypasses the lock check. Because market-authorized burns are separately exposed via `burnFromMarket` (onlyMarket), allowing user burn-like transfers during lock period is inconsistent with the lock intent. The practical impact is mostly self-harm (user destroys own tokens) and supply distortion, so severity is low.

1. Lock condition excludes `to == address(0)` path

```
function _update(address from, address to, uint256 id, uint256 value)
    internal
    virtual
    override(ERC6909, ERC6909TokenSupply)
{
    if (from != address(0) && to != address(0) && !_transfersUnlocked()) {
        revert IOOutcomeToken.TransfersLockedUntilGraduationOrResolution();
    }

    super._update(from, to, id, value);
}
```

When locked, `from != 0 && to != 0` is false if `to == 0`, so no revert.

2. Dedicated market-only burn path exists

```
function burnFromMarket(address from_, uint256 outcomeId_, uint256 amount_)
    external
    onlyMarket
{
    _burn(from_, outcomeId_, amount_);
}
```

The contract already models burns as a privileged market operation.

3. Why this is not high/medium impact

This does not let an attacker steal third-party funds directly; it primarily allows a holder to destroy their own locked position early. It can skew token supply for that outcome (which affects per-token payout math), but the burner pays the cost. The strongest effect is protocol/UX inconsistency with transfer lock semantics.

Impact

- This does not let an attacker steal third-party funds directly; it primarily allows a holder to destroy their own locked position early. It can skew token supply for that outcome (which affects per-token payout math), but the burner pays the cost. The strongest effect is protocol/UX inconsistency with transfer lock semantics.
- Lock policy is bypassable for burn-like operations before graduation/resolution. Users can self-burn during the lock window, creating avoidable supply distortion and confusing payout optics.

Recommendations

Block user-initiated burn path while locked, while preserving market-authorized burns. Example pattern: if `from != address(0)` and `!_transfersUnlocked()`, allow only the market's controlled burn flow (or revert unconditionally for all user-originated updates).

Status

Fixed

[I-03] `redeem()` makes redundant cross-contract call for winner balance

Description

In `redeem()` on `Market.sol`, for resolved markets, `OutcomeToken.balanceOf(msg.sender, correctOutcome)` is called twice - once for the zero-balance check and again for the burn amount. No state changes occur between the two reads. The second call is an unnecessary cross-contract `staticcall` that wastes gas.

Recommendation

Cache the first read and reuse it for both the check and the burn.

Recommendations

Cache the first read and reuse it for both the check and the burn.

Status

Fixed

[I-04] `positionCount` increments on `burnCompleteSet`, making it an operation counter rather than a position counter

Description

`positionCount` is documented as “Total number of positions/trades placed” and returned via `getMarketData()`. It increments in `buyOnPhase1`, `mintCompleteSet` and `burnCompleteSet`:

```
// burnCompleteSet – Market.sol
_recordActivityTimestamp();
positionCount++; // increments on a CLOSE, not a new position
```

A complete set burn is the exit of an existing position, not a new one. A user who mints and immediately burns a complete set generates `positionCount += 2`, while having no net position. Any off-chain consumer using `positionCount` to measure market participation or depth will over-report activity.

Recommendation

Remove `positionCount++` from `burnCompleteSet`. If tracking all lifecycle operations (opens + closes) is intentional, rename the variable to `operationCount` and update the NatSpec accordingly.

Recommendations

Remove `positionCount++` from `burnCompleteSet`. If tracking all lifecycle operations (opens + closes) is intentional, rename the variable to `operationCount` and update the NatSpec accordingly.

Status

Fixed

[I-05] Extremely low tick/size combinations can produce zero-notional matches (EDGE CASE)

Description

CLOB notional is computed with integer floor division: `(amount * priceBps) / 10_000`. If `amount * priceBps < 10_000`, the result is zero. Matching then transfers tokens while charging/transferring zero collateral and zero fees for that fill. This requires very small `minOrderSize` and very low `minTick` /price configurations, but it is mathematically reachable under permissive parameters.

1. Notional math uses floor division

```
function _getNotional(uint256 amount, uint256 priceBps)
    internal
    pure
    returns (uint256)
{
    return (amount * priceBps) / BPS_DENOMINATOR;
```

2. Matching consumes this notional directly

```
uint256 notional = _getNotional(matchAmount, askPrice);
uint256 takerFee = _getFee(notional, takerFeeBps);
uint256 makerRebate = _getFee(notional, makerRebateBps);

filledAmount += matchAmount;
spentNotional += notional;
paidTakerFee += takerFee;
```

If `notional == 0`, trade size advances but spend/fees do not.

3. Config bounds do not guarantee strictly positive notional

```
function _validateLimitOrderInputs(uint256 priceBps, uint256 amount)
    internal
    view
{
    if (amount == 0) revert IOrderBook.ZeroAmount();
    if (amount < minOrderSize) revert IOrderBook.AmountTooSmall();
    if (priceBps < minTick || priceBps > maxTick) {
        revert IOrderBook.InvalidPriceTick();
    }
}
```

And factory only enforces non-zero minima, not `minOrderSize * minTick >= 10_000`.

Impact

- Under extreme low-price/low-size settings, takers can obtain tiny token quantities at zero collateral/fee notional per fill. Usually negligible and bounded by config choices, but it weakens pricing integrity.

Recommendations

Add a config invariant (factory and/or CLOB init) such as `minOrderSize * minTick >= BPS_DENOMINATOR`, or enforce `notional > 0` per executed fill with a revert/skip strategy.

Status

Fixed

Conclusion

This security audit of the AmericanSpend identified 30 vulnerabilities across 3 High, 10 Medium, 12 Low, 5 Informational. All issues have been addressed by the development team.

Key findings addressed include:

- **High:** 3 high findings were identified and addressed
- **Medium:** 10 medium findings were identified and addressed
- **Low:** 12 low findings were identified and addressed
- **Informational:** 5 informational findings were identified and addressed

Key improvements implemented include:

- Enhanced input validation and access controls
- Improved error handling and edge case coverage
- Strengthened security patterns and best practices

The development team's commitment to addressing these findings demonstrates a strong security-focused approach to smart contract development. The protocol has undergone significant improvements based on the audit findings, with all reported issues resolved. Continued monitoring and periodic security reviews are recommended as the protocol evolves.

This report was prepared by 33Labs and represents our independent security assessment of the AmericanSpend project.